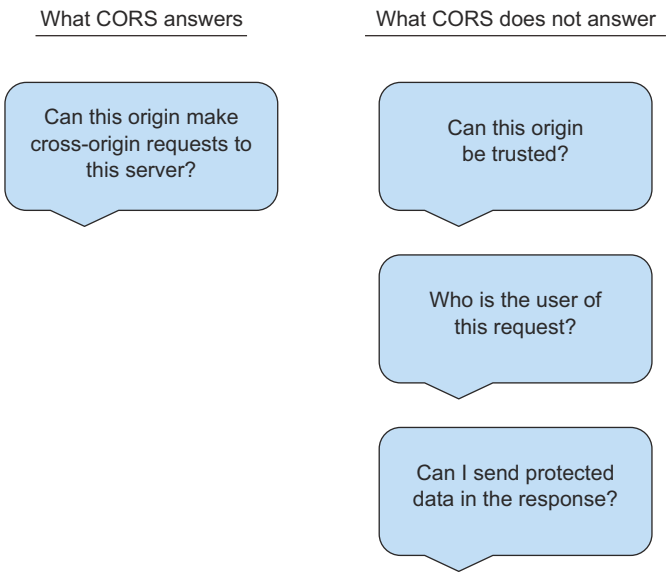


Figure 6.10 Questions CORS does and doesn't answer

This is an important enough point to state again: CORS isn't security. The Access-Control-Allow-Origin header only indicates which clients are allowed to make cross-origin requests. It shouldn't also be used to protect a site's content.

Figure 6.10 shows some questions that CORS does and doesn't answer. Don't be disheartened if you see a question that you'd like answered. Just because CORS doesn't answer a particular question doesn't mean it can't be answered at all. There are other mechanisms to answer these questions, which this section will explore.

To demonstrate why CORS shouldn't be used for security, consider that it's possible to use tools to spoof the Origin header. The following curl command shows how to send a request to your API with the Origin header set to `somerandomorigin.com` (if you don't know what curl is, don't worry; it will be covered in the next chapter).

```
curl -H "Origin: somerandomsite.com" http://127.0.0.1:9999/api/posts
```

This request is obviously not coming from somerandomsite.com, and your server shouldn't trust only the Origin header. This is why the Access-Control-Allow-Origin header should only be used to specify the cross-origin policy.

Spoofing CORS requests

The preceding curl request may have set off a red flag: What is the point of CORS if an Origin header can be spoofed? To answer this, it's important to keep in mind that users can use tools like curl to make an HTTP request to any server, regardless of whether or not the server has CORS enabled. CORS doesn't introduce any new security holes; it only ensures that cross-origin requests from a browser are performed in a safe manner.